

Summary of LLM Inference 2/19/2026

Minkyu Jun (mkjun), Kevin Jia (kevinjia), Matt Schumacher (ummatt), Maddy Heyer (mmheyer)

NanoFlow: Towards Optimal Large Language Model Serving Throughput

Problem and Motivation

Modern LLM serving systems are commonly assumed to be memory-bound due to the large KVCache and model sizes. However, NanoFlow shows that end-to-end LLM serving is actually compute-bound for most realistic workloads and models.

Despite this, existing systems (i.e. **vLLM**, **DeepSpeed**, **TensorRT-LLM**) achieve only 22%-38% of theoretical optimal throughput, because they execute compute-bound, memory-bound, and network-bound operations sequentially on GPUs.

This sequential execution causes:

- Pipeline bubbles where GPU compute units sit idle
- Poor overlap between compute, memory, and network operations
- Underutilization of GPU FLOPs capacity
- Reduced throughput and higher serving costs

Thus, the key inefficiency that NanoFlow presents is not insufficient hardware, but poor intra-device resource utilization due to lack of fine-grained parallelism.

Related Works

Prior work on improving LLM efficiency focused on two ways: improving serving throughput and operation level parallelism.

For throughput optimization, **Orca** employs request-level continuous batching to maximize batch size per iteration, while **DistServe** and **Splitwise** separate prefill and decode phases across clusters. In addition to that, **DeepSpeed-FastGen** and **Sarathi-Serve** introduce chunked prefill, splitting requests into smaller chunks to improve overall utilization.

For the optimization of operation level parallelism, **Rammer** and **Unity** exploit intra-operation parallelism and algebraic transformations. **ASPEN** and **Welder** construct tile-level data graphs to overlap operations.

Based on the works mentioned above, NanoFlow leverages fine-grained intra-device parallelism, integrates new operations through performance profiling, and increases overlapping opportunities via nano-batching, enabling more efficient and flexible execution.

Solution Overview

In NanoFlow, the authors improve LLM serving throughput by exploiting intra-device parallelism within a single GPU. Instead of executing operators sequentially, NanoFlow overlaps heterogeneous operations, such as compute-bound and memory-bound kernels, by splitting each iteration into smaller nano-batches and running corresponding nano-operators concurrently on different fractions of GPU resources. This reduces pipeline bubbles and increases hardware utilization.

To make overlapping safe and effective, NanoFlow builds an empirical performance model. It first profiles individual kernels to understand how runtime scales with reduced resource allocation, then measures pairwise kernel interference to capture slowdown under co-execution. Using these measurements, it performs a two-phase automated pipeline search:

- **Phase I:** constructs an ideal interference-free schedule that selects the number, size, and ordering of nano-operations to maximize overlap,
- **Phase II:** refines this schedule by incorporating interference-aware runtime estimates to minimize end-to-end latency.

Finally, NanoFlow includes a request scheduler that maintains dense token batches, prioritizes ongoing decode requests, and continuously feeds the optimized pipeline to sustain high GPU utilization. Together, these techniques enable higher serving throughput without requiring additional hardware.

Limitations

- At low request rates, NanoFlow experiences higher latency than other models because its parallel mechanism becomes more effective only when enough nano-batches accumulate to minimize pipeline bubbles.
- The paper focuses mostly on latency/throughput while there's less information about TTFT, or TBT which would provide a more complete evaluation.
- Nano-batches and overlap do not seem to provide the majority of the performance increase. The performance increase might have come from different sectors such as GPU memory management or scheduling policy
- The paper talks about network-bound work and even gives them a separate class of operators, but the motivation behind it is slightly lackluster. Only when running on older hardware and PCIe did problems ever show up

Future Research Directions

1. NanoFlow relies on interleaving prefill and decode stages on a single device. However, other modern techniques such as NVIDIA Dynamo and Mooncake recommend disaggregating these stages. Future work may explore how to construct pipelines similar to the ones they present in NanoFlow that are specifically tailored to the prefill or decode stages.
2. A key tradeoff that NanoFlow makes is higher latency in exchange for higher utilization. Another avenue for future research is to create multiple optimal pipelines for different batch sizes, similar to the way Oobleck creates different templates for different numbers of GPUs. This may reduce latency by reducing the time required to “fill” batches.
3. The version of NanoFlow the authors present in the paper is primarily concerned with intra-device parallelism. However, LLM inference providers often use multi-device configurations to serve requests. Future work may explore how to implement and then measure the effect of NanoFlow’s parallelism in different multi-device setups.

Summary of Class Discussion

(* involves both papers)

***How would you combine NanoFlow and Mooncake?:** It's a little weird to combine the two. Mooncake requires the disaggregation of the prefill/decode stages, whereas NanoFlow leverages the difference between the prefill and decode operations to fill in pipeline bubbles. Therefore, it doesn't really make sense to combine them. There is also less benefit in their joint integration because transferring of the KVCache on the granularity of nanobatches may be too excessive.

NanoFlow mentions a profiling stage. How often is it run? NanoFlow runs the profiler once before the pipeline is deployed. In the paper, the authors note that the optimizer required about ten minutes to complete their evaluations. If the configuration were to change, the optimizer, and subsequently the profiler, would have to be run again.

How does NanoFlow prevent having too many nano-operations or nano-batches in the overlapping strategy?: The short answer is that they use mixed integer linear programming, which the authors don't go too much into detail. However, this question is worthwhile to consider because if too many operators exist, you might lose some of the batching effect. Additionally, each batch may launch a kernel, which may begin overwhelming if the batches become too small

In Nanoflow, if there aren't enough tokens to fill a batch, does it just wait for it to fill?: A key assumption that Nanoflow makes is that there is always a surplus of requests, so such a scenario shouldn't really occur? However, it would be pretty simple to add another threshold, perhaps based on time spent waiting, and jump the gun on executing the batch.

Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot

Problem and Motivation

In conventional LLM serving systems, both Prefill (the phase for KVCache generation) and Decoding are often bottlenecked by memory capacity rather than compute. GPU VRAM can fill quickly with KVCache, especially for long-context models, leaving compute units idle. This is exacerbated when requests with overlapping prefixes in different instances cannot reuse KVCache from other instances due to local storage limitations, resulting in redundant prefill computation and increased Time-To-First-Token (TTFT). Traditional systems also tightly couple Prefill and Decoding on the same instance, limiting parallelism. Mooncake proposes to solve this problem by utilizing CPU, DRAM, and SSD to store more cached values and enabling cross-CPU networks to share cache information between Prefill and Decode instances in a cluster.

Related Works

Prior work related to improving LLM serving efficiency focuses on three directions: scheduling, memory and KVCache management, and resource disaggregation.

Product systems such as **FasterTransformer**, **TensorRT-LLM**, and **DeepSpeed Inference** primarily improve throughput through highly optimized inference runtimes. **Orca** introduces iteration-level scheduling to enhance concurrency, while **vLLM** improves memory efficiency via dynamic KVCache management.

FlexGen, **Sarathi-Serve**, and **FastServe**, explore scheduling and memory swapping strategies to better utilize limited hardware resources. In addition, prefix caching techniques aim to reuse KVCache across requests. **Prompt Cache** precomputes and stores frequently used KV states, and **SGLang** employs RadixAttention with an LRU-based radix tree to enable efficient cache sharing.

Building on these mechanisms, Mooncake further advances optimization through a prefill and decode disaggregation architecture.

Solution Overview

Mooncake introduces a disaggregated KVCache-centric architecture:

1. **CPU / DRAM / SSD KVCache storage:** Each node in Prefill instances streams KVCache to its CPU DRAM or SSD, reducing GPU memory pressure and enabling storage of much larger KVCache (up to tens of millions of tokens).

2. **Cluster-wide KVCache sharing:** Instances can retrieve KVCache generated by other prefill nodes via the Mooncake Store (which resides in CPU memory across nodes for cross-instance sharing), avoiding redundant prefill computation and increasing cache hit rates.
3. **Asynchronous KVCache transfers:** Prefill sends KVCache layer-wise to CPU memory, overlapping computation and communication. Decoding instances copies KVCache from CPU memory on demand through RDMA (Remote Direct Memory Access), minimizing idle time and latency.
4. **Prefill / Decoding separation:** Prefill and Decoding are executed on separate instances, enabling pipeline parallelism. Prefill continues generating KVCache while Decoding nodes consume previously completed KVCache for token generation.

This design allows the system to scale KVCache storage beyond the limits of local GPU VRAM and fully exploit reuse opportunities across requests with shared prefixes.

Building on this architecture, Mooncake also addresses the scheduling challenge of assigning requests to batches and instances efficiently. To do this, it introduces a global scheduler called Conductor, which is KVCache-aware and significantly more sophisticated than traditional load-based scheduling.

The scheduling mechanism/strategies follows below:

1. **Prefill node selection:** Conductor evaluates each prefill instance based on current load, estimated prefill execution time, local queue time.
2. **Execution time prediction:** Prefill execution time for a request is predicted using a polynomial regression model trained on offline measurements, accounting for request length, prefix match length, and KVCache transfer time.
3. **Prioritization:** Requests are effectively scheduled in ascending order of TTFT (shortest to longest), ensuring maximum KVCache utilization while balancing overall cluster load.
4. **Early rejection for SLO compliance:** Requests unlikely to meet TTFT or token-by-token latency (TBT) SLOs are preemptively rejected to maintain system performance under overload.
5. **Global cache-aware balancing:** Hot KVCache blocks (Cache that are used frequently) are replicated across multiple nodes to reduce remote fetch latency.

The key insight of Mooncake lies in fully exploiting CPU resources as a scalable extension of GPU memory. CPU DRAM provides significantly larger storage capacity than GPU VRAM, enabling the system to store substantially more KVCache. More importantly, RDMA-enabled cross-node CPU memory access allows KVCache sharing across instances at a cost that is often lower than recomputing the cache on GPUs. This shifts the trade-off from recomputation toward remote reuse, significantly improving system goodput.

Beyond architectural disaggregation, Mooncake also represents a fundamental shift from request-centric scheduling to KVCache-centric scheduling. The scheduler optimizes for cache reuse and expected TTFT rather than simply balancing load, enabling simultaneous improvements in throughput and latency in large-scale deployments.

Limitations

There are several limitations that the paper did not fully clarify:

1. The paper uses RDMA which implies instances in Mooncake are physically close to each other. However, in practice, nodes or instances in a cluster may not be physically co-located, which can introduce heterogeneous network latency and bandwidth. Such variability could potentially reduce the effectiveness of the proposed architecture, particularly for latency-sensitive KV-cache accesses. It is also important to note that Mooncake requires RDMA hardware support.
2. While the paper discusses the use of both DRAM and SSD in CPU, it does not analyze performance sensitivity to the DRAM–SSD capacity ratio. Given the substantial latency gap between SSD and DRAM, a more detailed evaluation of how different tiering configurations affect throughput and latency would strengthen the study.
3. While the Mooncake architecture is well suited for long-context workloads with high KV reuse, its benefits for short-query workloads are less clear. When sequences are short, limited cache reuse may cause the overhead of storing and managing KV cache in CPU memory to outweigh the performance gains, potentially reducing efficiency.

Future Research Directions

1. More research on the short-context workload environment may be helpful since the paper doesn't address it. If Mooncake results in worse performance compared to other prior mechanisms in such a situation, it would be great to propose alternative solutions for the short-context queries.
2. Network bandwidth is assumed to be high in the paper, but it might not be the case in the real world situation. In order to maximize the Mooncake's performance, it would be interesting to figure out which nodes/instances should be included in the cluster to maximize the connection speed.
3. Current systems may drop requests when SLOs cannot be met. In practice, workloads are bursty and diverse, so a more robust system should dynamically choose among reuse, recomputation, partial reuse, or graceful degradation. Investigating effective fallback mechanisms for these scenarios would improve real-world applicability.

Summary of Class Discussion

Cost of KV cache – diverse prompts that do not reuse the KV cache heavily: the KV cache hit rate reported in the paper is fairly high regardless of how diverse the prompt is. This can be considered credible because the evaluation uses real-world scenarios (Conversation, Tool & Agent, Synthetic). However, overall performance can degrade when the KV cache hit rate falls below its typical level.

SLO handling – If the SLO is not met, the system abandons the request. Should we actually abandon the request completely, or should we still run it even if performance is worse?: Typically, developers would not choose to drop requests entirely. In real-world systems, completely dropping

requests is uncommon because it worsens user experience. It is important to find a balance between how often full drops occur and the resulting performance loss so that users are not noticeably impacted.

DRAM vs. SSD – The paper does not mention the cache ratio between DRAM and SSD. Additional research may be needed to determine whether SSD is worthwhile and whether prefetching is necessary to reduce potential SSD I/O overhead.

Physical locality of instances – The paper mentions high communication bandwidth, which may imply close physical locality or a high-performance hardware setup.